Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Cloud Computing
Data 2026/4/13 Class AI 1
Name Weijie Xu ID 202383910029

一、Experimental purpose

(1) To understand the basic architecture of Kubernetes by setting up a local cluster using Minikube and exploring its core components.
(2) To become familiar with essential Kubernetes tools, including Minikube and kubectl, and learn how to interact with a cluster through command-line operations.
(3) To deploy a containerized application in Kubernetes by creating a Deployment, thereby understanding how applications are managed and scheduled.
(4) To observe the lifecycle and status of Pods, and analyze how Kubernetes ensures application availability and stability.
(5) To expose an application using a Service and access it externally, gaining insight into Kubernetes networking and service abstraction mechanisms.
(6) To explore the Kubernetes Dashboard and optional addons, in order to understand cluster monitoring and extensibility features.
(7) To develop hands-on experience with the complete workflow of application deployment, management, and access within a Kubernetes environment.

二、Experimental Contents

- **Minikube Cluster Initialization**

A local Kubernetes cluster was initialized using Minikube, with the Docker driver configured to ensure compatibility on a Windows environment. The cluster startup process included downloading necessary images and setting up core components such as the API server and kubelet. This step verified that the Kubernetes environment was correctly established and ready for subsequent operations.

- **Cluster Status Verification and Configuration Inspection**

The status of the Minikube cluster was checked using command-line tools to ensure that all components were running properly. Configuration details were examined through kubectl, allowing an understanding of how the client communicates with the cluster. This process helped confirm the correctness of the environment setup and connectivity.

- **Application Deployment via Kubernetes Deployment**

A containerized application was deployed by creating a Deployment resource using kubectl. This step involved specifying the container image and execution parameters,

enabling Kubernetes to manage the creation and scheduling of Pods. The deployment process demonstrated how applications are defined declaratively and maintained automatically.

- **Pod Monitoring and Log Analysis**

The created Pods were monitored to observe their lifecycle states, including initialization and running status. Logs generated by the application were retrieved and analyzed to verify that the service was functioning as expected. This step provided insight into debugging and runtime monitoring within Kubernetes.

- **Service Creation and External Access Configuration**

A Service resource was created to expose the deployed application to external users. By mapping internal container ports to accessible endpoints, the application could be accessed through a browser. This process illustrated the abstraction of networking in Kubernetes and how services enable stable communication with dynamic Pods.

- **Application Access and Functional Verification**

The exposed service was accessed using Minikube tools, which provided a URL for browser-based interaction. Successful access confirmed that the deployment and networking configurations were correctly implemented. This step validated the end-to-end workflow from deployment to user interaction.

三、Detailed steps of the experiment

The local environment has been successfully configured with the required Kubernetes tools. The kubectl client is installed with version v1.34.1, along with Kustomize v5.7.1 for resource configuration management. In addition, Minikube is properly installed with version v1.38.1, ensuring support for creating and managing a local Kubernetes cluster. These versions indicate that the system is fully prepared to proceed with container deployment and cluster-based application experiments.

```
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\cloud-computing-course\Labs\Lab03> kubectl version --client
Client Version: v1.34.1
Kustomize Version: v5.7.1
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\cloud-computing-course\Labs\Lab03> minikube version
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0
```

The Docker environment is currently running without any active containers, as indicated by the empty output of the docker ps command. This suggests that no containers have been started or are in execution at this stage, providing a clean initial state for subsequent container deployment and Kubernetes-related operations.

```
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\cloud-computing-course\Labs\Lab03> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

The Minikube cluster was successfully initialized on a Windows 11 system using the Docker driver. During the startup process, the required Kubernetes components and base images were automatically downloaded and configured, including the control

plane and networking interface. A Docker container was created to host the cluster, with allocated CPU and memory resources, and the necessary addons such as the storage provisioner were enabled. The Kubernetes environment was then verified, and kubectl was properly configured to interact with the default Minikube cluster and namespace, indicating that the local cluster is fully operational and ready for application deployment.

```
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\cloud-computing-course\Labs\Lab03> minikube start
😄 Microsoft Windows 11 Pro 25H2 上的 minikube v1.38.1
✨ 根据用户配置使用 docker 驱动程序
❗ Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
😄 Microsoft Windows 11 Pro 25H2 上的 minikube v1.38.1
✨ 根据用户配置使用 docker 驱动程序
😄 Microsoft Windows 11 Pro 25H2 上的 minikube v1.38.1
✨ 根据用户配置使用 docker 驱动程序
😄 Microsoft Windows 11 Pro 25H2 上的 minikube v1.38.1
😄 Microsoft Windows 11 Pro 25H2 上的 minikube v1.38.1
✨ 根据用户配置使用 docker 驱动程序
❗ Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
📌 使用具有 root 权限的 Docker Desktop 驱动程序
👍 在集群中 "minikube" 启动节点 "minikube" primary control-plane
🚜 正在拉取基础镜像 v0.0.50 ...
💾 正在下载 Kubernetes v1.35.1 的预加载文件...
    > gcr.io/k8s-minikube/kicbase...:  519.58 MiB / 519.58 MiB  100.00% 14.13 M
    > preloaded-images-k8s-v18-v1...:  272.45 MiB / 272.45 MiB  100.00% 5.43 Mi
🔥 创建 docker container（CPU=2，内存=3500MB）...-
🐳 正在 Docker 29.2.1 中准备 Kubernetes v1.35.1…|
🔗 配置 bridge CNI (Container Networking Interface) ...
🔎 正在验证 Kubernetes 组件...
    ▪ 正在使用镜像 gcr.io/k8s-minikube/storage-provisioner:v5
🌟 启用插件： storage-provisioner, default-storageclass
💡 kubectl 未找到。如果你需要使用它，请尝试：'minikube kubectl -- get pods -A'
🏄 完成！kubectl 现在已配置，默认使用"minikube"集群和"default"命名空间
```

The Minikube cluster is currently running in a healthy state, with all core components functioning correctly. The control plane has been successfully initialized, and both the host and kubelet are in the running state. In addition, the API server is active, ensuring proper communication within the cluster, while the kubeconfig has been correctly configured to allow seamless interaction through kubectl. This indicates that the Kubernetes environment is fully operational and ready for further deployment and testing tasks.

```
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\cloud-computing-course\Labs\Lab03> minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

The application was successfully deployed in the Kubernetes cluster by creating a Deployment named hello-node. Initially, the deployment showed a pending state with no available replicas, which indicates that the Pod was still being initialized. Shortly afterward, the deployment reached a ready state with one available replica,

confirming that the application had been successfully scheduled and started. The corresponding Pod is currently running without any restarts, demonstrating that the containerized application is stable and functioning correctly within the cluster.

```
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\cloud-computing-course\Labs\Lab03> kubectl create deployment
 hello-node --image=registry.k8s.io/e2e-test-images/agnhost:2.53 -- /agnhost netexec --http-port=8080
deployment.apps/hello-node created
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\cloud-computing-course\Labs\Lab03> kubectl get deployments
NAME         READY   UP-TO-DATE   AVAILABLE   AGE
hello-node   0/1     1            0           8s
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\cloud-computing-course\Labs\Lab03> kubectl get deployments
NAME         READY   UP-TO-DATE   AVAILABLE   AGE
hello-node   1/1     1            1           58s
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\cloud-computing-course\Labs\Lab03> kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
hello-node-64fc5894d-9nf8g    1/1     Running   0          77s
```

The application inside the Pod has started successfully, as indicated by the log output. The container initialized both an HTTP server on port 8080 and a UDP server on port 8081, demonstrating that the service is actively running and capable of handling network requests. The absence of error messages further suggests that the application is functioning normally within the Kubernetes environment.

```
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\cloud-computing-course\Labs\Lab03> kubectl logs hello-node-6
4fc5894d-9nf8g
I0413 06:57:18.066205       1 log.go:245] Started HTTP server on port 8080
I0413 06:57:18.089416       1 log.go:245] Started UDP server on port  8081
```

The application was successfully exposed by creating a Service of type LoadBalancer, enabling external access to the deployed workload. The service named hello-node has been assigned a cluster-internal IP address and is listening on port 8080, which is mapped to a dynamically allocated node port.

```
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\cloud-computing-course\Labs\Lab03> kubectl expose deployment
 hello-node --type=LoadBalancer --port=8080
service/hello-node exposed
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\cloud-computing-course\Labs\Lab03> kubectl get services
NAME         TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
hello-node   LoadBalancer   10.105.167.199   <pending>     8080:30893/TCP   8s
kubernetes   ClusterIP      10.96.0.1        <none>        443/TCP          3m56s
```

The deployed application was successfully accessed through the Minikube service, which provided a URL for external interaction. The service exposed the application on port 8080 and mapped it to a dynamically assigned local address, allowing it to be opened directly in a web browser. This confirms that the networking configuration is functioning correctly and that the application can be reached from the host environment. The successful access demonstrates the complete workflow from deployment to external service availability within the Kubernetes cluster.

```
(base) PS D:\我的\大三\大三下\云计算导论\实验\石磊实验\cloud-computing-course\Labs\Lab03> minikube service hello-node
```

NAMESPACE	NAME	TARGET PORT	URL
default	hello-node	8080	http://192.168.49.2:30893

```
为服务 hello-node 启动隧道。
```

NAMESPACE	NAME	TARGET PORT	URL
default	hello-node		http://127.0.0.1:62578

```
正通过默认浏览器打开服务 default/hello-node...
因为你正在使用 windows 上的 Docker 驱动程序，所以需要打开终端才能运行它。
```

四、Reflection

1. Through this experiment, a clearer understanding of the overall Kubernetes workflow was achieved, including cluster initialization, application deployment, service exposure, and external access. The process helped connect theoretical concepts with practical operations, forming a more complete view of how Kubernetes manages containerized applications. Frequent use of kubectl and Minikube commands significantly improved familiarity with command-line interactions.

2. The experiment highlighted the abstraction mechanisms in Kubernetes, particularly the roles of Deployment, Pod, and Service. It became evident that Kubernetes separates application definition, execution, and access, which enhances flexibility and scalability in real-world scenarios. By checking Pod status and analyzing logs, practical experience in identifying and resolving issues was gained.

3. During the setup process, dependencies such as Docker, drivers, and system configurations were found to play a crucial role. Minor misconfigurations could lead to failures, highlighting the need for careful environment preparation and validation. The creation and exposure of Services provided practical insight into how Kubernetes handles networking. The difference between internal cluster communication and external access became clearer, especially in the context of Minikube.

附录 程序命令代码

附录 1：检查版本
kubectl version --client

```
minikube version

docker ps
```

附录 2：启动 Minikube 集群
minikube start minikube kubectl -- get pods -A minikube status

附录 3：创建 Kubernetes Deployment
kubectl create deployment hello-node --image=registry.k8s.io/e2e-test-images/agnhost:2.53 -- /agnhost netexec --http-port=8080 kubectl get deployments kubectl get pods kubectl logs hello-node-64fc5894d-9nf8g

附录 4：访问服务
kubectl expose deployment hello-node --type=LoadBalancer --port=8080 kubectl get services minikube service hello-node